

ShopEasy Reflection Report

This reflection summarizes the testing work performed for the ShopEasy term project, applying Chapters 1-6 techniques from Effective Software Testing. It explains the bugs discovered by each testing technique, compares their effectiveness, situates the suite within the testing pyramid, and describes what would be tested next with one more week.

The first technique, specification-based testing, focused on `PriceCalculator`. Writing tests that exercise valid partitions, boundaries, and invalid inputs helped expose the contract assumptions of the calculator. This task revealed that the implementation needed strong precondition validation, so I added assertions in `PriceCalculator.calculate(...)` and updated the spec tests to expect assertion failures for invalid rates. The most valuable lesson from this task was that clear domain partitions and boundary values are enough to find incorrect assumptions in a helper class, even when the formula itself looks simple.

The second technique, structural testing and code coverage, targeted `ShoppingCart`. By exercising add, remove, update quantity, discount, and total calculations, the branch coverage report showed whether every behavior path was tested. This task was effective for finding missing branches in the cart implementation and for validating edge cases such as removing a non-existent item, applying discounts on an empty cart, and ensuring `updateQuantity` rejected invalid values. The final shopping cart tests achieved the required branch coverage threshold and confirmed that the cart behaves correctly across its main operations.

The third technique, design by contract, required adding assertion-based preconditions and postconditions to production code and writing contract tests for valid and invalid inputs. This was especially powerful because it made assumptions explicit in code and forced the test suite to validate them. The assertions in `ShoppingCart.addItem()`, `ShoppingCart.applyDiscount()`, and `PriceCalculator.calculate()` capture the intended contracts directly. The contract tests then verified both that valid inputs pass and that invalid inputs fail fast with `AssertionError`. This approach caught cases that simple functional tests might accept silently, and it helped enforce implementation invariants such as `total() >= 0`.

The fourth technique, property-based testing with jqwik, was used for both `PriceCalculator` and `ShoppingCart`. I defined properties such as non-negative final price, zero discount and tax identity, discount monotonicity, and cart addition commutativity. The custom `@Provide` method generated valid `Product` objects for the cart tests. These properties are valuable because they express broad invariants over many inputs rather than individual examples. In this project, jqwik confirmed that the price calculator formula works correctly across a wide space and that cart totals are order-independent when adding distinct products.

The fifth technique, mocks and stubs, isolated `OrderProcessor` by replacing `InventoryService` and `PaymentGateway` with Mockito mocks. This made it possible to test the orchestration logic in four distinct scenarios: happy path, inventory failure, payment failure, and partial inventory failure. Without mocking, these tests would either require real inventory/payment systems or would be less reliable and slower. Mocking allowed the suite to verify that `OrderProcessor` correctly aborts on inventory failure and never charges the customer when inventory is insufficient.

Across all techniques, the most effective method per unit of effort was the combination of structural testing and design by contract. Structural testing provided precise coverage feedback and identified missing operational paths, while the contract assertions made implicit assumptions explicit. Together they created a strong foundation for both example-based and property-based testing.

The ShopEasy suite is best classified as a classic unit-heavy test suite in the testing pyramid. Most of the work is at the unit level: direct tests of `PriceCalculator`, `ShoppingCart`, and `OrderProcessor` logic. The property-based tests are also unit-level, even though they explore a wider input space, and the mock-based tests keep dependencies isolated. There are no full end-to-end integration tests in this project, so the suite is solid on the bottom of the pyramid but would benefit from more integration coverage if the application grew larger.

If I had one more week, I would focus on two areas. First, I would add mutation testing to measure how well the tests detect real code changes, especially around `ShoppingCart` and `OrderProcessor`. Surviving mutants would reveal weak assertions and missing branching conditions. Second, I would add a few integration-style tests that exercise `OrderProcessor` with a simple real inventory implementation and a stubbed payment gateway, to complement the unit and mock-based checks with higher-level behavior validation.

In summary, the ShopEasy testing work applied the required techniques in sequence, with each task strengthening the next. Specification-based testing established the domain rules, structural testing verified implementation branches, design by contract made invariants explicit, property-based testing confirmed broad behavioral properties, and mocks enabled isolated orchestration testing. The remaining improvement opportunity is to add mutation testing and a small number of integration tests for even greater confidence.